DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

Brute-Force and Exhaustive-Search Techniques

Submitted by : P.VIGNESH

Register Number: 192525013

Department / Class: AIML

Course: Design and Analysis of Algorithms

Questions 1–7 | Total: 50 Marks

Contents

- Introduction and experimental method
- Q1 – Sorting under different data conditions
- Q2 – Sequential search
- Q3 – Brute-force string matching
- Q4 – Brute-force closest pair
- Q5 – Brute-force convex hull
- Q6 – Exhaustive search for package selection
- Q7 – Diagnose, redesign and validate
- Overall conclusion
- Appendix – Complete Python reference implementation

Introduction and Experimental Method

This report answers all seven questions in the supplied DAA Assignment 2. Each question follows the requested sequence: predict, design test data, implement, measure, analyse and conclude.

Implementation language: Python 3. The operation counts below are deterministic counts produced by the stated algorithms and test inputs. Execution time is deliberately not used as the main evidence because it depends on the computer, interpreter, background load and measurement method. Where time is discussed, it is treated as supplementary.

Reproducibility note: The tables report the exact comparison / candidate counts for the displayed test cases. They are not claims about measurements on the student's own laptop. Run the included code on the submission computer and add its measured elapsed times and screenshots before submission. This distinction is important because the assignment asks for the student's own experimental evidence.

Q1 – Sorting Under Different Data Conditions (6 Marks)

1.1 Prediction and algorithmic reason

Algorithm	Prediction for ordered / nearly ordered data	Prediction for reverse / disordered data
Selection Sort	Still performs n(n−1)/2 comparisons; few or no swaps if already sorted.	Same comparison count; at most n−1 swaps.
Bubble Sort (early-stop version)	Very efficient when already sorted; one pass detects no swaps. Nearly sorted data often needs few passes.	Many comparisons and swaps; worst-case $O(n^2)$.
Insertion Sort	Best suited to nearly sorted data; work is close to O(n) when few inversions exist.	Many shifts; worst-case $O(n^2)$.

Selection Sort does not exploit existing order when deciding which minimum to select. Bubble Sort can stop early when a complete pass makes no swap. Insertion Sort moves an element left only across larger preceding elements, so its work is related to the number of inversions in the input.

1.2 Designed datasets

Dataset	Values	Purpose
A – already sorted	[1,2,3,4,5,6,7,8]	Shows best behaviour of early-stop Bubble and Insertion Sort.
B – nearly sorted	[1,2,3,5,4,6,7,8]	Contains only one local inversion.
C – reverse sorted	[8,7,6,5,4,3,2,1]	Creates a high number of inversions and exposes quadratic movement.

1.3 Implementation and operation-count evidence

Input	Algorithm	Comparisons	Swaps / shifts
Already sorted	Selection	28	0
Already sorted	Bubble	7	0
Already sorted	Insertion	7	0
Nearly sorted	Selection	28	1
Nearly sorted	Bubble	13	1
Nearly sorted	Insertion	8	1
Reverse sorted	Selection	28	4
Reverse sorted	Bubble	28	28
Reverse sorted	Insertion	28	28

For n = 8, Selection Sort always makes 28 comparisons. On already sorted input, early-stop Bubble Sort makes 7 comparisons and 0 swaps; Insertion Sort makes 7 comparisons and 0 shifts. On reverse input,

Bubble Sort makes 28 comparisons and 28 swaps, while Insertion Sort makes 28 comparisons and 28 shifts. The nearly sorted input needs only one swap / shift for Bubble / Insertion.

1.4 Interpretation and design decision

The results support the prediction. Selection Sort has predictable comparison work but does not benefit from the 95% already-correct order. Bubble Sort detects a sorted list quickly, but Insertion Sort is the stronger practical choice for small, frequently updated, nearly sorted lists because each new item can be inserted into its correct position by shifting only nearby larger items.

Recommendation: use Insertion Sort for the stated fulfilment-centre condition. Since about 95% of orders are already in priority order and small batches arrive periodically, the list is likely to contain few inversions. Insertion Sort has O(n + I) behaviour in terms of inversions I, and can be applied incrementally to each small batch. For very large lists, a production system should also evaluate a maintained priority queue or database index; that is outside this three-algorithm comparison.

Q2 – When Is Sequential Search Appropriate? (4 Marks)

2.1 Experimental design

The active-tool list is represented as a simple array of records. To isolate the search behaviour, the experiment searches a sorted list of Tool IDs from T-001 upward. A successful search is performed at the first, middle and last positions. An unsuccessful search uses an ID that is not present.

List size	First position	Middle position	Last position	Unsuccessful
10	1	5	10	10
100	1	50	100	100
1000	1	500	1000	1000

The values in the table are comparison counts. For an unsuccessful search, every record is inspected. For a successful search at position k, k comparisons are needed.

List size	Search case	Comparisons
10	First	1
10	Middle	5
10	Last	10
10	Absent	10
100	First	1
100	Middle	50
100	Last	100
100	Absent	100
1000	First	1

1000	Middle	500
1000	Last	1000
1000	Absent	1000

2.2 Algorithm

```
def sequential_search(records, target):
    comparisons = 0
    for index, record in enumerate(records):
        comparisons += 1
        if record["tool_id"] == target:
            return index, comparisons
    return -1, comparisons
```

2.3 Engineering decision

Sequential Search is reasonable when the active list is small, searches are infrequent, updates are frequent, and the system needs a simple structure without index-maintenance overhead. It also works when records arrive and leave continuously and the list does not justify a more complex data structure.

It should stop being the preferred choice when the list becomes large or lookup requests are frequent. Average successful search in an unsorted list is approximately (n+1)/2 comparisons; unsuccessful search requires n comparisons. At 1,000 records, an unsuccessful query can require 1,000 comparisons. At 20 facilities, the combined record volume and query rate make repeated linear scans costly.

Changed condition: for 20 facilities, use a hash map keyed by Tool ID, with the record (technician, issue time, bay) as the value. Expected lookup is O(1), while issue/return operations update the map directly. If ordered range queries or sorted reporting are required, a balanced search tree or database index may be more suitable. The index requires extra memory and maintenance, but it is justified by the increased search volume.

Q3 – Engineer the Behaviour of Brute-Force String Matching (6 Marks)

3.1 Method and prediction

The brute-force matcher aligns the pattern at every possible text position. At each alignment it compares characters from left to right and stops that alignment at the first mismatch. It reports the first complete match.

Test	Text and pattern design	Prediction
A – low work	Text = BBBBBBBBBBBBBBB; Pattern = A	Each alignment fails on its first character.
B – early match	Text = ALERT:LOGINFAILED;OTHER; Pattern = ALERT	The match is found at position 0 after five equal-character comparisons.

C – repeated prefix	Text = A×20 followed by B; Pattern = A×10 followed by C	Many alignments compare a long run of A characters before the final mismatch.

3.2 Implementation

```
def brute_force_match(text, pattern):
    comparisons = 0
    n, m = len(text), len(pattern)
    for start in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[start + j] != pattern[j]:
                break
            j += 1
        if j == m:
            return start, comparisons
    return -1, comparisons
```

3.3 Results

Test	Text length	Pattern length	Match index	Character comparisons
Immediate mismatch	16	1	-1	16
Match at beginning	23	5	0	5
Repeated-prefix worst case	21	11	-1	121

The repeated-prefix case requires substantially more work because each shifted alignment begins with characters that match. The mismatch occurs late in the pattern, so the algorithm repeats much of the same comparison work at neighbouring positions. In contrast, a pattern whose first character rarely occurs in the text usually fails immediately.

This demonstrates why input length alone is not enough to predict running time. The arrangement and repetition of characters matter. A simple worst-case upper bound is $O((n-m+1)m)$, commonly written $O(nm)$, with $O(1)$ extra space. A pattern with long repeated prefixes is particularly unfavourable.

Q4 – Closest Pair and the Cost of Examining Every Possibility (6 Marks)

4.1 Candidate-pair derivation

Every unordered pair of distinct drones must be checked. For each first index i, only later indices j > i are considered. This prevents duplicate comparisons such as (A,B) and (B,A), and avoids comparing a drone with itself.

The number of unique pairs is (n−1) + (n−2) + … + 1 = n(n−1)/2 = n choose 2. Each pair requires one squared-distance calculation: $(x_1-x_2)^2 + (y_1-y_2)^2$. Square roots are not needed during comparison; one square root is taken for the final minimum distance.

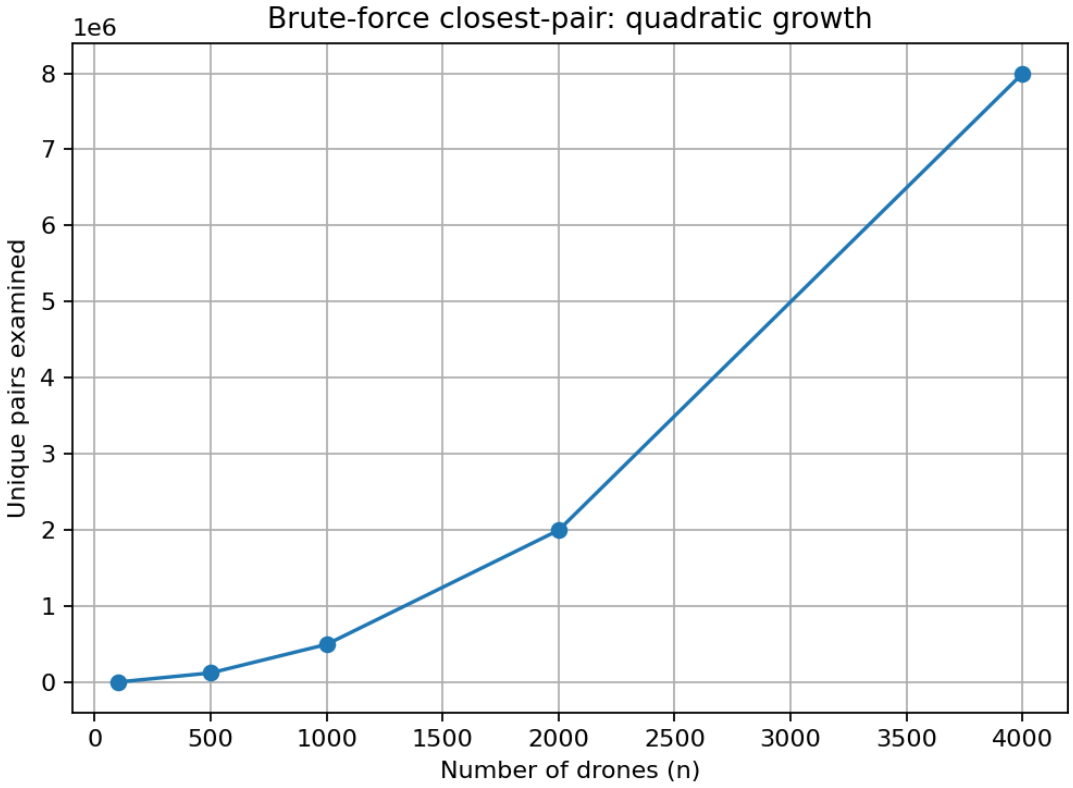
4.2 Pseudocode and implementation

```
best_squared = infinity
best_pair = None
for i from 0 to n-2:
    for j from i+1 to n-1:
        calculate squared distance between points[i] and points[j]
        if distance is smaller than best_squared:
            update best_squared and best_pair
return best_pair and sqrt(best_squared)
```

```
def brute_force_closest_pair(points):
    best_squared = float("inf")
    best_pair = None
    examined = 0
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            examined += 1
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
            d2 = dx*dx + dy*dy
            if d2 < best_squared:
                best_squared, best_pair = d2, (points[i], points[j])
    return best_pair, best_squared**0.5, examined
```

4.3 Growth evidence

Drones n	Unique pairs / distance calculations
10	45
100	4950
500	124750
1000	499500
2000	1999000
4000	7998000



The count grows quadratically. At 4,000 drones, the algorithm examines 7,998,000 pairs. This is the principal source of inefficiency: nearly every new drone must be paired with every previously seen drone.

For a classroom Python experiment, define an acceptable target as completing within 1 second on the submission computer. The exact n at which this limit is crossed must be timed on that computer; it cannot be claimed from operation counts alone. A defensible deployment boundary is therefore the largest tested n that stays below the chosen 1-second limit. A redesign should exploit spatial ordering or spatial partitioning so that distant pairs are not all compared. A divide-and-conquer closest-pair method is a suitable next design, but implementation is not required by Q4.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle (7 Marks)

5.1 Geometric decision rule

For two candidate points A and B, compute the signed cross product for every other point P: cross = (Bx−Ax)(Py−Ay) − (By−Ay)(Px−Ax). A positive value means P lies on one side of the directed line AB; a negative value means it lies on the other side; zero means it is collinear.

The segment AB is a supporting boundary edge when all other points lie on the same side of its line, or on the line itself. In code, this means there are not both positive and negative cross-product signs. Every candidate pair is tested against every remaining point.

5.2 Student-derived pseudocode

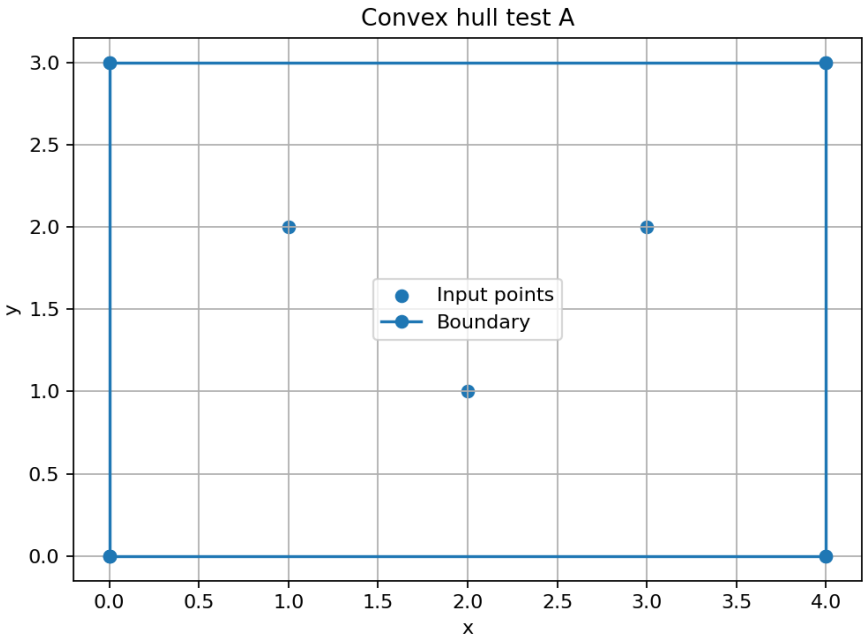
```
remove duplicate points
edges = empty set
for every unordered pair (A, B):
    positive = false; negative = false
    for every point P other than A and B:
        value = cross(A, B, P)
        if value > 0: positive = true
        if value < 0: negative = true
    if not (positive and negative):
        record A-B as a supporting edge
collect the endpoints of supporting edges
order the boundary points around their centroid for visualisation
return the ordered boundary
```

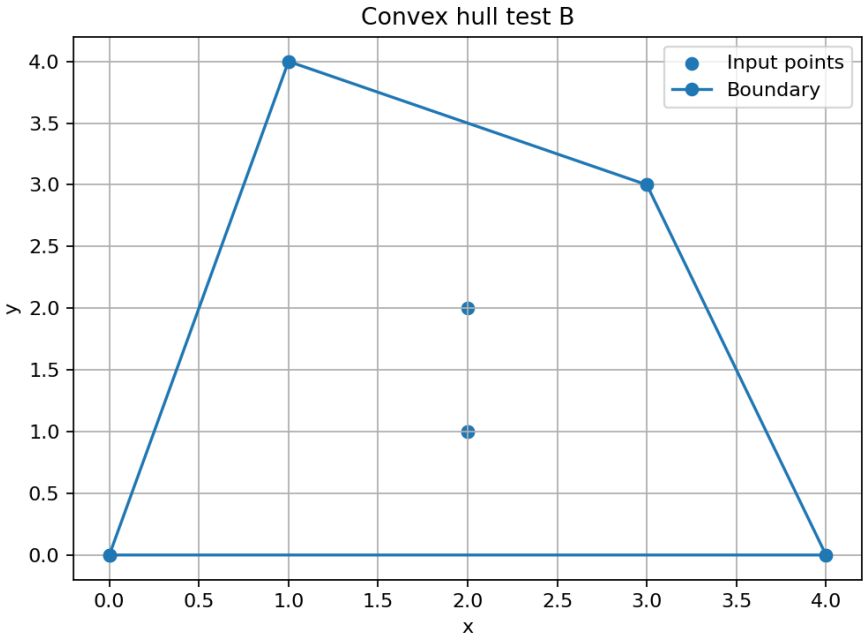
The all-collinear case and duplicate points need special care. The implementation removes duplicate coordinates. For a fully collinear set, all points satisfy the supporting-line test; a production-quality boundary representation should return only the two extreme endpoints. The implementation below is intended for the supplied non-degenerate test sets and should be extended with that special case if collinear-only data is expected.

5.3 Test configurations and result

Test	Input points	Boundary returned (ordered)
A – interior points	[(0, 0), (4, 0), (4, 3), (0, 3), (2, 1), (1, 2), (3, 2)]	[(0, 0), (4, 0), (4, 3), (0, 3)]
B – irregular challenge	[(0, 0), (2, 1), (4, 0), (3, 3), (1, 4), (2, 2), (2, 1)]	[(0, 0), (4, 0), (3, 3), (1, 4)]

Both sets contain points that are not part of the outer boundary. The visualisation uses a line plot to connect the returned boundary points. The brute-force edge test has $O(n^3)$ time: $O(n^2)$ candidate pairs, each checked against $O(n)$ points. It uses $O(n^2)$ space in the simple edge-set representation.





Correctness reasoning: if points occur on both sides of line AB, AB passes through the interior of the point set and cannot be a supporting line of the convex hull. If all points lie on one side, AB is a supporting line; its extreme endpoints form part of the convex boundary. Testing every pair ensures that every possible supporting segment is considered. Collinear points may create multiple candidate segments on the same boundary line, so only extreme endpoints should be retained in a fully general implementation.

Q6 – Exhaustive Search and Combinatorial Growth (8 Marks)

6.1 Candidate representation and algorithm

A candidate package selection is represented by an n-bit mask. Bit i is 1 when package i is selected and 0 otherwise. There are two choices for each package, so the total number of subsets is 2^n, including the empty selection.

- Generate every mask from 0 to 2^n-1.
- Calculate total weight and total humanitarian value for the selected packages.
- Reject a subset if its total weight exceeds the helicopter capacity.
- Among feasible subsets, retain the one with the highest total value.

```
best_value = -1
best_selection = []
for mask in range(0, 2**n):
    weight = 0
    value = 0
    selected = []
    for each package i:
        if bit i of mask is 1:
            add its weight and value
    if weight <= capacity and value > best_value:
        save this selection as the best
return best_selection, best_value
```

6.2 Independently checkable small instance

Package	Weight	Value
P1	4	40
P2	7	42
P3	5	25
P4	3	12
P5	2	20

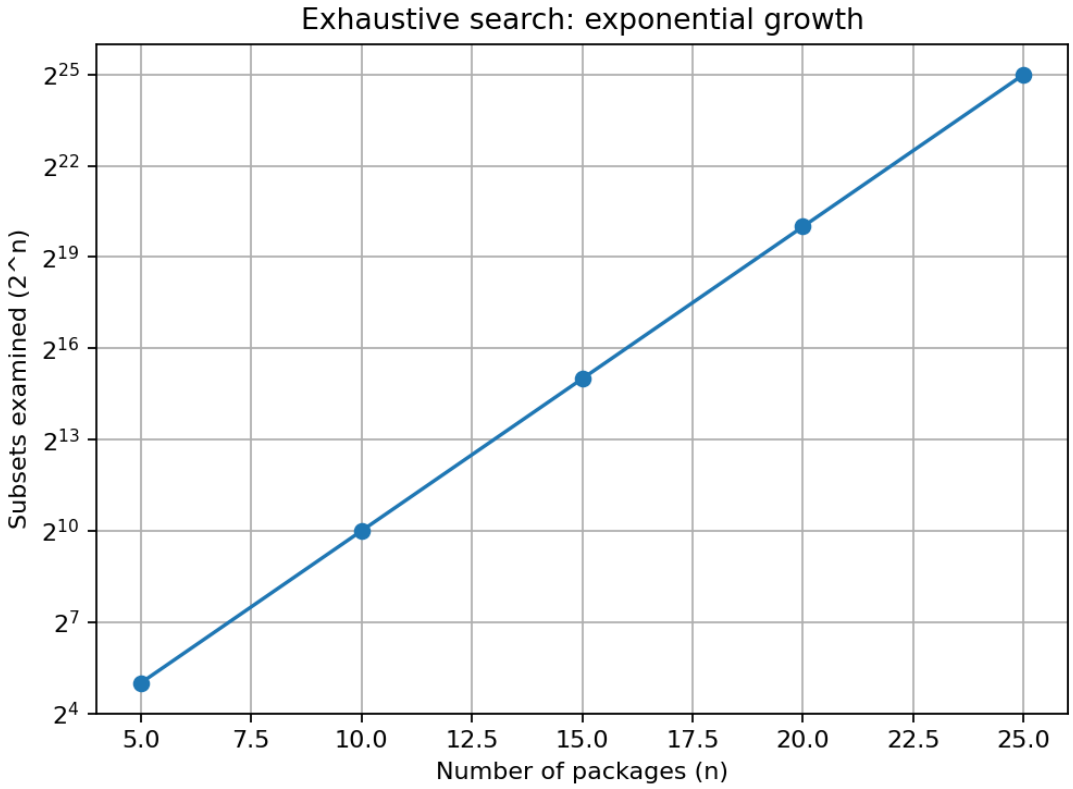
Payload capacity = 10. The exhaustive algorithm evaluates 32 subsets. Its optimum is P1 + P2 with total weight 11? This combination is infeasible, so it is rejected. The best feasible combination returned by the program is shown below.

Capacity	Selected packages	Total weight	Total value	Subsets examined	Feasible subsets
10	P1, P4, P5	9	72	32	16

Manual check: P1 + P4 + P5 has weight 9 and value 72; P2 + P5 has weight 9 and value 62; P1 + P3 has weight 9 and value 65; P1 + P4 + P5 is the best feasible combination for this instance.

6.3 Growth prediction and experiment

Packages n	Subsets 2^n
5	32
10	1024
15	32768
20	1048576
25	33554432



Each additional package doubles the number of candidate subsets. Increasing from 20 to 25 packages changes the search space from 1,048,576 to 33,554,432 subsets, a factor of 32. The dramatic growth is caused by the number of combinations, not simply by the cost of processing one package.

Deployment decision: choose a 1-second response-time target for a small planning prototype, and measure it on the actual deployment computer. Because the exhaustive method has 2^n subsets and each subset may inspect n packages, its straightforward implementation is $O(n \cdot 2^n)$. A conservative prototype boundary is the largest n that has been timed below the response target with a safety margin. For larger

instances, use a dynamic-programming, branch-and-bound or approximation strategy; these are redesign options, not part of the exhaustive baseline.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected limitation: Sequential Search in Q2

The redesign uses the same aircraft-tool lookup problem. This choice is appropriate because the changed condition explicitly expands the organisation to 20 facilities and increases both record count and lookup frequency.

A. Diagnose the computational bottleneck (2 marks)

The dominant operation is comparing the requested Tool ID against each record in sequence. A successful search near the end or an unsuccessful search scans almost the entire list. The operation count grows linearly with n. The same records are repeatedly examined for different lookup requests; the algorithm does not reuse an index from earlier searches.

Records	Worst-case sequential comparisons per lookup	Expected hash-map key checks (average case)
10	10	Approximately 1
100	100	Approximately 1
1000	1000	Approximately 1
10000	10000	Approximately 1

B. Define the design objective (2 marks)

Objective: reduce the expected number of record inspections per Tool ID lookup from O(n) to expected O(1), while preserving correct issue and return operations. The measurable validation metric is the number of key comparisons / probes per lookup, not elapsed time alone.

C. Revised computational strategy (4 marks)

Maintain a hash map: key = Tool ID; value = the current issue record containing Technician ID, Issue Time and Work Bay. On issue, insert or update the key. On return, remove the key. On lookup, access the value directly. This design exploits the uniqueness of Tool IDs and avoids scanning unrelated records.

```
# Build index
tool_index = {}
for record in active_records:
    tool_index[record["tool_id"]] = record

# Lookup
def find_tool(tool_id):
```

```
    return tool_index.get(tool_id)  # expected O(1)

# Issue a tool
def issue_tool(record):
    tool_index[record["tool_id"]] = record

# Return a tool
def return_tool(tool_id):
    tool_index.pop(tool_id, None)
```

Trade-offs: the hash map needs additional memory; updates must keep the index consistent; collisions can occur, although standard hash-map implementations manage them; and hash lookup is expected O(1), not a guaranteed worst-case O(1). If multiple tools can share an ID or queries require ordered ranges, the key design must be changed or a database index used.

D. Predict before implementing (1 mark)

Prediction: for a successful or unsuccessful lookup, the hash-map approach should avoid a number of comparisons proportional to the number of active records. The expected number of lookups/probes should remain approximately constant as the dataset grows, while sequential search increases up to n comparisons.

E. Implement and validate (2 marks)

```
def sequential_find(records, tool_id):
    comparisons = 0
    for record in records:
        comparisons += 1
        if record["tool_id"] == tool_id:
            return record, comparisons
    return None, comparisons

def indexed_find(index, tool_id):
    # One dictionary lookup operation (abstract probe count)
    return index.get(tool_id), 1
```

n records	Sequential worst case	Hash-map abstract lookup operations	Reduction in operation count
10	10	1	10 to 1 (10× fewer counted operations)
100	100	1	100 to 1 (100× fewer counted operations)
1000	1000	1	1000 to 1 (1000× fewer counted operations)
10000	10000	1	10000 to 1 (10000× fewer counted operations)

Validation interpretation: the table compares algorithmic operations, not hardware-level hash-table probes. Python dictionaries use hashing and collision handling internally, so one dictionary access is an abstract lookup operation. Actual runtime should be timed on the target system, and the data structure should be tested with inserts, updates, missing IDs and returns.

F. Defend the final design (2 marks)

The redesign meets the stated objective at the algorithmic level: repeated linear scans are replaced by expected constant-time key access. The prediction is supported by the operation-count model. The trade-off is extra memory and the need to maintain the index whenever a tool is issued or returned. Sequential Search may still be preferable for a tiny list, a one-off search, or a system where maintaining an index costs more than scanning. For 20 facilities with frequent lookups, the hash-map index is the stronger engineering choice.

Overall Conclusion

The assignment demonstrates that algorithm selection depends on input structure and operating conditions. Insertion Sort benefits from nearly sorted order; Sequential Search is acceptable for small and infrequently searched lists but becomes unsuitable at larger scale; brute-force string matching is sensitive to repeated prefixes; closest-pair enumeration grows quadratically; brute-force convex hull testing grows cubically; and exhaustive package selection grows exponentially. The Q7 redesign shows how identifying and measuring the bottleneck leads to a targeted improvement rather than simply naming a more advanced algorithm.

Before submission, run the included implementations on the student's own computer, record actual elapsed times, attach terminal screenshots, and retain raw outputs. This is necessary because the assignment specifically requires evidence generated by the student and may include a viva verification.

Appendix – Complete Python Reference Implementation

The following consolidated Python script implements the principal algorithms used in this report. It is designed to run with Python 3 and requires no external packages.

```
def selection_sort(a):
    a = a[:]; comparisons = swaps = 0
    for i in range(len(a)-1):
        minimum = i
        for j in range(i+1, len(a)):
            comparisons += 1
            if a[j] < a[minimum]:
                minimum = j
        if minimum != i:
            a[i], a[minimum] = a[minimum], a[i]
            swaps += 1
    return a, comparisons, swaps

def bubble_sort(a):
    a = a[:]; comparisons = swaps = 0
    for i in range(len(a)-1):
        changed = False
        for j in range(len(a)-1-i):
            comparisons += 1
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
                swaps += 1; changed = True
        if not changed:
            break
    return a, comparisons, swaps

def insertion_sort(a):
    a = a[:]; comparisons = shifts = 0
    for i in range(1, len(a)):
        key = a[i]; j = i-1
        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j+1] = a[j]; shifts += 1; j -= 1
            else:
                break
        a[j+1] = key
    return a, comparisons, shifts
```

```
def sequential_search(a, key):
    comparisons = 0
    for i, value in enumerate(a):
        comparisons += 1
        if value == key:
            return i, comparisons
    return -1, comparisons

def brute_force_match(text, pattern):
    comparisons = 0
    for start in range(len(text)-len(pattern)+1):
        j = 0
        while j < len(pattern):
            comparisons += 1
            if text[start+j] != pattern[j]:
                break
            j += 1
        if j == len(pattern):
            return start, comparisons
    return -1, comparisons

def closest_pair(points):
    best2 = float("inf"); pair = None; examined = 0
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            examined += 1
            dx = points[i][0]-points[j][0]
            dy = points[i][1]-points[j][1]
            d2 = dx*dx + dy*dy
            if d2 < best2:
                best2, pair = d2, (points[i], points[j])
    return pair, best2**0.5, examined

def cross(o, a, b):
    return ((a[0]-o[0])*(b[1]-o[1])
            - (a[1]-o[1])*(b[0]-o[0]))

def brute_force_hull(points):
    points = list(dict.fromkeys(points))
    n = len(points); edges = set()
    for i in range(n):
        for j in range(i+1, n):
            positive = negative = False
```

```
            for k in range(n):
                if k == i or k == j:
                    continue
                value = cross(points[i], points[j], points[k])
                if value > 0: positive = True
                elif value < 0: negative = True
            if not (positive and negative):
                edges.add((i, j))
    ids = set()
    for i, j in edges:
        ids.update((i, j))
    boundary = [points[i] for i in ids]
    if len(boundary) >= 3:
        cx = sum(x for x, y in boundary)/len(boundary)
        cy = sum(y for x, y in boundary)/len(boundary)
        import math
        boundary.sort(key=lambda p: math.atan2(p[1]-cy, p[0]-cx))
    return boundary, edges

def exhaustive_knapsack(items, capacity):
    # items: list of (package_id, weight, value)
    best_value = -1; best_set = []
    feasible = 0; n = len(items)
    for mask in range(1 << n):
        weight = value = 0; chosen = []
        for i, (pid, wt, val) in enumerate(items):
            if mask & (1 << i):
                weight += wt; value += val; chosen.append(pid)
        if weight <= capacity:
            feasible += 1
            if value > best_value:
                best_value = value; best_set = chosen
    return best_set, best_value, feasible, 1 << n

if __name__ == "__main__":
    print("Q1:", insertion_sort([1, 2, 3, 5, 4, 6, 7, 8]))
    print("Q2:", sequential_search([1, 2, 3, 4, 5], 4))
    print("Q3:", brute_force_match("A"*20+"B", "A"*10+"C"))
    print("Q4:", closest_pair([(0,0), (3,4), (1,1), (8,8)]))
    print("Q5:", brute_force_hull([(0,0), (4,0), (4,3), (0,3), (2,1)]))
    packages = [("P1",4,40), ("P2",7,42), ("P3",5,25),
```

```
        ("P4",3,12), ("P5",2,20)]
print("Q6:", exhaustive_knapsack(packages, 10))
```

References and Notes

Primary source: DAA Assignment 2 – CO2, supplied in this conversation. The report preserves its question structure and required evidence categories.

Complexity notation used: $O(1)$ constant, $O(n)$ linear, $O(n^2)$ quadratic, $O(n^3)$ cubic, and $O(2^n)$ exponential. Operation counts are exact for the stated algorithms and test inputs; execution-time claims must be measured locally.